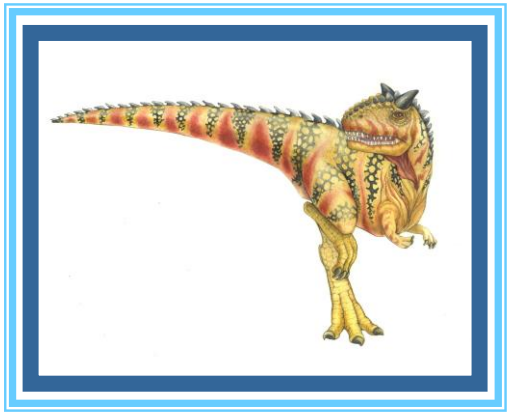
Chapter 7: Synchronization Examples



Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
  - Bounded-Buffer Problem
  - Readers and Writers Problem
  - Dining-Philosophers Problem

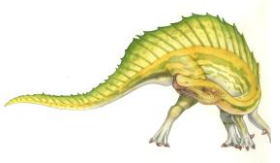
Bounded-Buffer Problem

- ***n*** buffers, each can hold one item
- Semaphore **`mutex`** initialized to the value 1
- Semaphore **`full`** initialized to the value 0
- Semaphore **`empty`** initialized to the value n

Bounded Buffer Problem (Cont.)

- The structure of the producer process

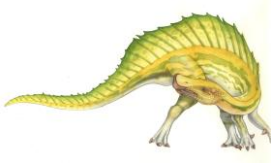
```
while (true) {
        ...
        /* produce an item in next_produced */
        ...
    wait(empty);
    wait(mutex);
        ...
        /* add next produced to the buffer */
        ...
    signal(mutex);
    signal(full);
}
```



Bounded Buffer Problem (Cont.)

- The structure of the consumer process

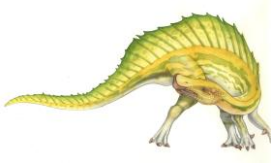
```
while (true) {
    wait(full);
    wait(mutex);
        ...
     /* remove an item from buffer to next_consumed */
        ...
    signal(mutex);
    signal(empty);
        ...
       /* consume the item in next consumed */
        ...
   }
```



Bounded-Buffer Problem

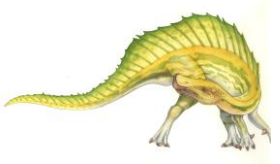
```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t mutex, empty, full;

int main() {
    pthread_t prod, cons;
    sem_init(&mutex, 0, 1);
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    sem_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

```c
void* producer(void* arg) {
    int item = 0;
    while (1) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = item++;
        in = (in + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&full);
        printf("Produced: %d\n", item - 1);
    }
    return NULL;
}

void* consumer(void* arg) {
    while (1) {
        sem_wait(&full);
        sem_wait(&mutex);
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        sem_post(&mutex);
        sem_post(&empty);
        printf("Consumed: %d\n", item);
    }
    return NULL;
}
```



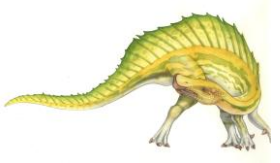
Readers-Writers Problem

- A data set is shared among a number of concurrent processes
  - **Readers** – only read the data set; they do ***not*** perform any updates
  - **Writers** – can both read and write
- Problem – allow multiple readers to read at the same time
  - Only one single writer can access the shared data at the same time
- Several variations of how readers and writers are considered – all involve some form of priorities



Readers-Writers Problem (Cont.)

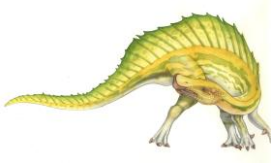
- Shared Data
  - Data set
  - Semaphore **`rw_mutex`** initialized to 1
  - Semaphore **`mutex`** initialized to 1
  - Integer **`read_count`** initialized to 0



Readers-Writers Problem (Cont.)

- The structure of a writer process

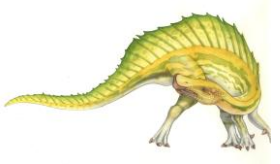
```
while (true) {
      wait(rw_mutex);
         ...
      /* writing is performed */
         ...
   signal(rw_mutex);
}
```



Readers-Writers Problem (Cont.)

- The structure of a reader process

```
while (true){
        wait(mutex);
        read_count++;
        if (read_count == 1) /* first reader */
             wait(rw_mutex);
        signal(mutex);
        ...
        /* reading is performed */
        ...
        wait(mutex);
        read_count--;
        if (read_count == 0) /* last reader */
              signal(rw_mutex);
        signal(mutex);
}
```



Readers-Writers Problem (Cont.)

```c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t rw_mutex, mutex;
int read_count = 0;

void* writer(void* arg) {
    while (1) {
        sem_wait(&rw_mutex);
        printf("Writer is writing\n");
        sem_post(&rw_mutex);
    }
    return NULL;
}
```

C program for the first readers-writers problem

```c
void* reader(void* arg) {
    while (1) {
        sem_wait(&mutex);
        read_count++;
        if (read_count == 1)
            sem_wait(&rw_mutex);
        sem_post(&mutex);
        printf("Reader %ld is rea
        sem_wait(&mutex);
        read_count--;
        if (read_count == 0)
            sem_post(&rw_mutex);
        sem_post(&mutex);
    }
    return NULL;
}
```

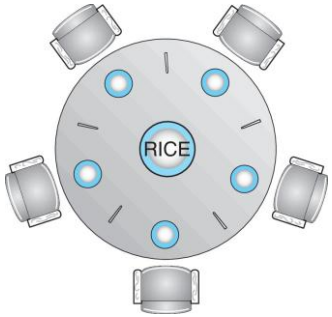
Readers-Writers Problem (Cont.)

C program for the first readers-writers problem

```
int main() {
    pthread_t w, r1, r2;
    sem_init(&rw_mutex, 0, 1);
    sem_init(&mutex, 0, 1);
    pthread_create(&w, NULL, writer, NULL);
    pthread_create(&r1, NULL, reader, (void*)1);
    pthread_create(&r2, NULL, reader, (void*)2);
    pthread_join(w, NULL);
    pthread_join(r1, NULL);
    pthread_join(r2, NULL);
    sem_destroy(&rw_mutex);
    sem_destroy(&mutex);
    return 0;
}
```

Dining-Philosophers Problem

- N philosophers' sit at a round table with a bowel of rice in the middle.



- They spend their lives alternating thinking and eating.
- They do not interact with their neighbors.
- Occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
  - Need both to eat, then release both when done
- In the case of 5 philosophers, the shared data
  - Bowl of rice (data set)
  - Semaphore chopstick [5] initialized to 1

Dining-Philosophers Problem Algorithm

- Semaphore Solution
- The structure of Philosopher i :

```
while (true){
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

     /* eat for awhile */

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );

     /* think for awhile */

}
```

- What is the problem with this algorithm?

Dining-Philosophers Problem Algorithm

How to solve deadlocks in Dining philosophers' problem?

Several possible remedies to the deadlock problem are the following:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up her chopsticks only if both chopsticks are available (to do this, she must pick them up in a critical section).
- Use an asymmetric solution—that is, an odd-numbered philosopher picks up first her left chopstick and then her right chopstick, whereas an even-numbered philosopher picks up her right chopstick and then her left chopstick.